Clearmark: Bookmark & Extension Manager (Requirements)

1. Overview

Clearmark is a browser extension that lets a user manage all of their bookmarks and all of their installed extensions from one place. It reads the browser's native bookmark tree, offers fast search and bulk actions, and includes an extension hub to list, enable, disable, and remove installed extensions, and to open store pages for the ones the user wants to install. It can also remove itself from its own settings page.

Primary target: Chrome (Manifest V3). Edge and Brave work with the same code. Firefox support is a later phase.

Goals

- One dashboard to view, search, organize, and clean up every bookmark.
- An extension hub to see every installed extension, turn them on or off, remove the ones the user no longer wants, and open the store page for new ones.
- Work directly on the browser's real bookmarks, so nothing is out of sync.
- Keep permissions minimal and the code simple to review.

Non-goals (v1)

- Installing other extensions silently (not possible, see section 8). The hub only opens store pages, and the user clicks "Add to Chrome" themselves.
- Cloud sync or user accounts.
- Mobile browsers.

2. Tech Stack

- Manifest V3
- TypeScript
- React with Vite, built to static files (no server needed at runtime)
- Tailwind CSS for styling
- `chrome.bookmarks`, `chrome.storage`, `chrome.runtime`, `chrome.management` (extension hub and self uninstall)
- Vitest for unit tests, Playwright for basic end to end checks
- Package manager: pnpm (npm is fine too)

Suggested build helper: `@crxjs/vite-plugin` or a plain Vite multi page config that outputs `dist/` ready for "Load unpacked".

3. Project Structure

```
clearmark/
  manifest.json
  package.json
  vite.config.ts
  tsconfig.json
  src/
    background/
      service-worker.ts       # listens to bookmark events, sets uninstall URL
    popup/
      index.html
      Popup.tsx               # quick search, add current page, open manager
    manager/
      index.html
      App.tsx                 # full page dashboard
      components/
        FolderTree.tsx
        BookmarkList.tsx
        SearchBar.tsx
        BulkActionBar.tsx
        DuplicatesPanel.tsx
        DeadLinksPanel.tsx
        ImportExport.tsx
        ExtensionHub.tsx      # list, enable, disable, remove, open store page
        ExtensionCard.tsx
        StoreLinks.tsx        # user's "want to install" list
    options/
      index.html
      Options.tsx             # settings and "Remove extension" button
    lib/
      bookmarks.ts            # typed wrapper around chrome.bookmarks
      storage.ts              # typed wrapper around chrome.storage
      duplicates.ts
      linkcheck.ts
      exportImport.ts
      extensions.ts           # typed wrapper around chrome.management
    types.ts
  public/
    icons/ (16, 32, 48, 128 png)
  tests/
```

4. Manifest Requirements

- `manifest_version: 3`
- Name, version, description, icons (16, 32, 48, 128)
- `action` with a popup
- `options_page` pointing to the options page
- `background.service_worker`
- `chrome_url_overrides` is NOT used (do not replace the new tab page)
- Permissions: `bookmarks`, `storage`, `tabs` (only to open the manager tab and read the current tab for "add current page"), `management` (only for the extension hub: list, enable, disable, and uninstall other extensions)
- Note: `management` makes Chrome show a stronger install warning ("manage your apps, extensions, and themes"). This is expected and must be explained in the store listing and privacy policy.
- Optional host permissions for dead link checking, requested at runtime only when the user starts a link check
- Self uninstall (`chrome.management.uninstallSelf()`) does not need the `management` permission by itself. It is only requested here because of the extension hub. Keep all `management` calls inside `lib/extensions.ts`.

Starter manifest:

```json
{
  "manifest_version": 3,
  "name": "Clearmark: Bookmark & Extension Manager",
  "version": "0.1.0",
  "description": "Manage all your bookmarks and browser extensions from one place.
  "permissions": ["bookmarks", "storage", "tabs", "management"],
  "optional_host_permissions": ["<all_urls>"],
  "action": { "default_popup": "src/popup/index.html" },
  "options_page": "src/options/index.html",
  "background": { "service_worker": "src/background/service-worker.ts", "type": "m
  "icons": { "16": "icons/16.png", "32": "icons/32.png", "48": "icons/48.png", "12
}
```

5. Functional Requirements

5.1 Bookmark browsing (must have)

- FR-1: Load the full bookmark tree with `chrome.bookmarks.getTree()` and show it as a folder tree on the left and a list of items on the right.
- FR-2: Show title, URL, favicon, folder path, and date added for each bookmark.
- FR-3: Sort by title, date added, or URL. Toggle list and compact view.
- FR-4: Live update when bookmarks change anywhere in the browser (listen to `onCreated`, `onRemoved`, `onChanged`, `onMoved`).

5.2 Search and filter (must have)

- FR-5: Instant search across title and URL as the user types (debounced, under 100 ms for 10,000 bookmarks).
- FR-6: Filter by folder, by domain, and by date range.
- FR-7: Keyboard shortcut to focus search (`/` on the manager page).

5.3 Editing and organizing (must have)

- FR-8: Create, rename, edit URL, move, and delete bookmarks and folders.
- FR-9: Drag and drop to move items between folders.
- FR-10: Multi select with checkboxes. Bulk move and bulk delete.
- FR-11: Confirm before any delete. Offer a 10 second "Undo" for the last delete by recreating the removed items.

5.4 Cleanup tools (should have)

- FR-12: Duplicate detection by normalized URL (lowercase host, strip trailing slash, strip common tracking params like `utm_*`). Let the user pick which copy to keep.
- FR-13: Dead link checker. Runs on user request only, with limited concurrency (for example 5 at a time), a timeout of 8 seconds per URL, and a progress bar. Results are marked OK, redirected, or broken. Needs optional host permission requested at runtime.
- FR-14: Find empty folders and offer to delete them.

5.5 Tags and notes (nice to have)

- FR-15: Add tags and a short note to a bookmark. The browser has no field for this, so store them in `chrome.storage.local`, keyed by bookmark id.
- FR-16: Filter and search by tag.
- FR-17: Clean up stored tags and notes when a bookmark is removed.

5.6 Import and export (should have)

- FR-18: Export all bookmarks to JSON (including tags and notes) and to the standard Netscape HTML bookmark format.
- FR-19: Import from JSON and from Netscape HTML. Show a preview with counts before applying. Imported items go into a new folder named "Imported <date>".

5.7 Popup (should have)

- FR-20: Quick search box with the top 10 results.
- FR-21: "Add current page" button with folder picker.
- FR-22: "Open full manager" button that opens the manager page in a new tab.

5.8 Settings and uninstall (must have)

- FR-23: Options page with theme (light, dark, system) and default sort.
- FR-24: A "Remove extension" button on the options page that calls:

```ts
try {
  await chrome.management.uninstallSelf({ showConfirmDialog: true });
} catch {
  // user cancelled, show a small "Uninstall cancelled" message
}
```

- FR-25: Before calling uninstall, warn the user that tags and notes stored by the extension will be deleted, and offer to export first.
- FR-26: Call `chrome.runtime.setUninstallURL(url)` in the service worker so a short feedback page opens after removal. The URL is configurable and can be left empty.
- FR-27: Browser bookmarks themselves are never deleted by uninstalling the extension. State this clearly in the UI.

5.9 Extension hub (must have)

- FR-28: List all installed extensions with `chrome.management.getAll()`. Show icon, name, version, description, enabled state, and install type (normal, development, admin, sideload). Hide themes and apps by default, with a toggle to show them.
- FR-29: Search and filter extensions by name and by state (enabled, disabled).
- FR-30: Enable or disable any extension with `chrome.management.setEnabled(id, bool)`. Disable the switch and show a "Managed by your organization" label when `mayDisable` is false.

- FR-31: Remove an extension with `chrome.management.uninstall(id, { showConfirmDialog: true })`. Chrome shows its own confirmation dialog every time. Handle cancel without errors. Extensions managed by policy cannot be removed, so hide the remove button for them.
- FR-32: Bulk disable and bulk remove. With bulk remove, Chrome asks for confirmation for each extension, so queue them one at a time and show progress. The user can stop the queue at any point.
- FR-33: Never include Clearmark itself in bulk actions. Its own removal lives only on the options page (FR-24).
- FR-34: Show the permission warnings for each extension (`getPermissionWarningsById`) in a details drawer, so the user can see what an extension can access before deciding to keep it.
- FR-35: Keep the list live by listening to `chrome.management.onInstalled`, `onUninstalled`, `onEnabled`, and `onDisabled`.
- FR-36: "Install list" for extensions the user wants. The user can add an entry by pasting a Chrome Web Store URL or extension ID, or by searching by name. Each entry has an "Open in store" button that runs `chrome.tabs.create({ url: "https://chromewebstore.google.com/detail/<EXTENSION_ID>" })`. The user then clicks "Add to Chrome" themselves. Entries are stored in `chrome.storage.local`.
- FR-37: Export and import the installed extension list as JSON (id, name, enabled state). On import, show which ones are missing and offer "Open in store" for each one. This helps set up a new machine, but the install click always stays with the user.
- FR-38: Before the first use of the hub, show a short explanation of why the `management` permission is needed and what Clearmark does and does not do with it.

6. Non Functional Requirements

- **Privacy:** No analytics, no remote calls, no data leaves the browser. The only network requests are the user started dead link checks. The extension list is never sent anywhere.
- **Performance:** Handle 20,000 bookmarks smoothly. Use list virtualization (for example `@tanstack/react-virtual`) in the bookmark list.
- **Security:** Strict CSP from MV3 defaults, no `eval`, no remote code, escape all bookmark titles and URLs when rendering, never render bookmark content as HTML.
- **Accessibility:** Full keyboard navigation, visible focus states, ARIA labels on icon buttons, sufficient color contrast.
- **Compatibility:** Latest two versions of Chrome and Edge. Keep browser API calls behind `lib/` wrappers so a Firefox port only touches those files.
- **Code quality:** TypeScript strict mode, ESLint, Prettier, no unused permissions.

7. Data Model

Bookmark metadata kept by the extension (in `chrome.storage.local`):

```ts
type BookmarkMeta = {
  tags: string[];
  note?: string;
};

type WantedExtension = {
  id: string;            // Chrome Web Store extension id
  name: string;
  addedAt: number;
};

type Storage = {
  meta: Record<string, BookmarkMeta>;   // key is the browser bookmark id
  wantedExtensions: WantedExtension[];  // the "install list" (FR-36)
  settings: {
    theme: "light" | "dark" | "system";
    defaultSort: "title" | "dateAdded" | "url";
    uninstallUrl?: string;
  };
};
```

Everything else (title, URL, folder, order) lives in the browser's own bookmark tree and is never duplicated.

8. Platform Limits to Respect

These are browser rules, not choices, and the code must not try to work around them.

- An extension cannot install other extensions silently. To suggest another extension, open its store page with `chrome.tabs.create({ url: "https://chromewebstore.google.com/detail/<EXTENSION_ID>" })` and let the user click "Add to Chrome".
- An extension can only uninstall itself, and the user must confirm in Chrome's dialog.
- Uninstalling other extensions needs the `management` permission and Chrome shows a confirmation dialog for each one. Disabling and enabling needs the permission but no dialog. Policy managed extensions cannot be disabled or removed (`mayDisable` is false).

- The extension list is only what is installed in the current browser profile. Clearmark cannot see other profiles or other browsers.
- Code cannot run at the moment of uninstall. Do any cleanup before calling `uninstallSelf`.
- MV3 service workers are not persistent. Keep state in storage, not in global variables.

9. Milestones

1. **Scaffold:** Vite + React + TypeScript project, manifest, builds to `dist/`, loads with "Load unpacked", popup and manager open.
2. **Core manager:** folder tree, list, live updates, search, sort (FR-1 to FR-7).
3. **Editing:** CRUD, drag and drop, bulk actions, undo (FR-8 to FR-11).
4. **Settings and uninstall:** options page, theme, self uninstall flow, uninstall URL (FR-23 to FR-27).
5. **Extension hub:** list, search, enable, disable, remove, permission details, live updates, install list with store links, export and import (FR-28 to FR-38).
6. **Cleanup tools:** duplicates, empty folders, dead links (FR-12 to FR-14).
7. **Import and export:** JSON and HTML (FR-18, FR-19).
8. **Tags and notes:** (FR-15 to FR-17).
9. **Polish and release:** icons, store screenshots, privacy policy, tests, zip for the Chrome Web Store.

10. Acceptance Criteria

- Loads in Chrome via "Load unpacked" with no console errors or manifest warnings.
- All bookmarks shown in the manager match the browser's own bookmark manager (`chrome://bookmarks`).
- Adding, editing, moving, or deleting a bookmark in either place shows up in the other within one second.
- Search returns results as the user types with no visible lag on a 10,000 bookmark test set.
- Delete always asks for confirmation, and undo restores the item in its original folder.
- Export then import round trips without losing titles, URLs, folders, tags, or notes.
- The extension hub lists every installed extension in the profile, and the list updates within one second when an extension is installed, removed, enabled, or disabled elsewhere.
- Disabling and enabling an extension from the hub works, and policy managed extensions show as locked.
- Removing an extension from the hub opens Chrome's confirmation dialog, removes it on confirm, and leaves it untouched on cancel. Clearmark never appears in bulk actions.

- "Open in store" opens the correct Chrome Web Store page, and nothing is installed without the user clicking "Add to Chrome".
- The "Remove extension" button opens Chrome's confirmation dialog, removes the extension on confirm, and does nothing harmful on cancel.
- Only the permissions listed in section 4 appear in the install prompt.

11. Publishing Notes

- Chrome Web Store: one time developer registration fee of $5. Prepare a 128px icon, at least one 1280x800 screenshot, a short description, and a privacy policy stating that no data is collected.
- Edge Add-ons and Firefox Add-ons are free to publish to. Firefox needs `browser_specific_settings` in the manifest and a check of the `browser.*` API differences.
- Justify every permission in the store submission form (`bookmarks`: read and manage bookmarks, `storage`: save tags, settings, and the install list, `tabs`: open the manager page and add the current page, `management`: show installed extensions and let the user enable, disable, or remove them from the hub). Expect a closer review because of `management`, so state clearly that nothing about the extension list leaves the browser.

12. Instructions for Claude Code

1. Read this file fully, then scaffold the project exactly as in section 3.
2. Work milestone by milestone and stop after each one so the result can be loaded in Chrome and checked against the acceptance criteria.
3. Keep all `chrome.*` calls inside `src/lib/` wrappers, and keep UI components free of direct browser API calls.
4. Do not add permissions beyond section 4 without asking. Keep every `chrome.management` call inside `src/lib/extensions.ts`.
5. Add unit tests for `duplicates.ts`, `exportImport.ts`, and URL normalization as they are written.
6. Add a short `README.md` with build steps, how to load the extension unpacked, and how to run tests.